D Y PATIL UNIVERSITY
NAVI MUMBAI

Subject Name : Python

Unit No: IV

Unit Name : I/O and File Handling using Python

Faculty Name :Prachi Abhijeet Surve

D Y PATIL UNIVERSITY
NAVI MUMBAI

Unit No.: 4

Faculty Name :Prachi Abhijeet Surve

Index

Lecture No & Topic Name	Slide No
Input functions Output Formatting,	
Reading and Writing Files	
Reading and Writing to Binary Files	
Reading and Writing to CSV Files	

D Y PATIL UNIVERSITY
NAVI MUMBAI

Lecture No: 1
Input functions

Input Functions in Python

Python provides built-in functions to take user input.

input() Function:

Used to take input from user (always returns string).

```
name = input("Enter your name: ")
print("Hello", name)
```

Converting Input Type

```
age = int(input("Enter age: "))
salary = float(input("Enter salary: "))
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

Command Line Input:

Using sys.argv

```
import sys
print("Script name:", sys.argv[0])
Print("First argument:", sys.argv[1])
```

Output:

python file.py 100

D Y PATIL
UNIVERSITY
NAVI MUMBAI

Output Formatting

1. **Using Comma**

```
name = "Swati"
age = 30
Print("Name:", name, "Age:", age)
```

2. **Using format() Method:**

```
name = "Swati"
age = 30
print("Name: {} Age: {}".format(name, age))
```

With positions:

```
print("Name: {0} Age: {1}".format(name, age))
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

3. Using f-Strings (Recommended):

```
name = "Swati"
age = 30
Print(f"Name: {name} Age: {age}")
```

4. Formatting Numbers

```
pi = 3.14159
print(f"{pi:.2f}")      # 2 decimal places
Print(f"{pi:10.2f}")    # width 10
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

File Handling

-->As part of the programming requirement, we have to store our data permanently for future purpose. For this requirement we should go for files.
-->Files are are very common permenant storage areas to store our data.

Types of Files:

-->There are 2-tyes of files.
1).Text Files
2).Binary Files

1).Text Files:

-->Usually we can use text files to store characters data.
Ex: abc.txt

2).Binary Files:

-->Usually we can use binary files to store binary data like images, video files,audio files etc...

D Y PATIL UNIVERSITY
NAVI MUMBAI

Opening a file:

- Before performing any operation(like read or write) on the file, first we have to open that file. For this we should use python's inbuilt function open().
- But the time of the open, we have to specify mode, which represents the purpose of opening file.
- **Syntax:**

  Variable_name=open(filename,mode)

  For ex: f= open ("hello.txt", "w+")

  - We declared the variable f to open a file named hello.txt. Open takes 2 arguments, the file that we want to open and a string that represents the kinds of permission or operation we want to do on the file
  - Here we used "w" letter in our argument, which indicates write and the plus sign that means it will create a file if it does not exist in library
  - The available option beside "w" are "r" for read and "a" for append and plus sign means if it is not there then create it

D Y PATIL UNIVERSITY
NAVI MUMBAI

Allowed modes in python

Mode	Description
r	Open an existing file for read operations. The file pointer is positioned at the beginning of the file. If the specified file does not exist then we will get FileNotFoundError. This is default mode.
w	Open an existing file for write operation. If the file already contains some data then it will be overridden. If the specified file is not already available then this mode will create the file.
a	Open an existing file for append operation. It won't override existing data. If the specified file is not already available then this mode will create a new file.
r+	To read and write data into the file. The previous data in the file will not be deleted. The file pointer is placed at the beginning of the file.
w+	To write and read data. It will override existing data.
a+	To append and read data from the file. It won't override existing data.
x	To open a file in exclusive creating mode for write operation. If the file already exists then we will get FileExistError.
Note: All the above modes are applicable for text files. If the above modes suffixed with 'b' then these represents for binary files. Ex: rb, wb, ab, r+b, w+b,a+b,xb	

Closing a File

Ex:

f=open("abc.txt“ ,"w")

We are opening abc.txt file for writing data.

After completing our operations on the file, it is highly recommended to close the file. For this we have to use close() function.

f.close()

D Y PATIL UNIVERSITY
NAVI MUMBAI

Various properties of File Object:

Once we opened a file and we got file object, we can get various details related to that file by using its properties.

name -->Name of the opened file.
mode -->Mode in which the file is opened.
closed -->returns boolean value indicates that file is closed or not.
readable() -->Returns a boolean value indicates that whether file is readable or not.
writable() -->Returns a boolean value indicates that whether file is writable or not.

Eg:

```
f=open("abc.txt","w")
print("File Name:",f.name)
print("File Mode:",f.mode)
print("Is File Readable:",f.readable())
print("Is File Writable:",f.writable())
print("Is File Closed:",f.closed)
f.close()
print("Is File Closed:",f.closed)
```

```
File Name: abc.txt
File Mode: w
Is File Readable: False
Is File Writable: True
Is File Closed: False
Is File Closed: True
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

Writing data to text file:

We can write character data to the text files by using the following 2-methods.

1) .write(str)
2) .writelines(list of lines)

Eg:

```
f=open("abcd.txt","w")
f.write("Info\n")
f.write("Software\n")
f.write("Solutions\n")
print("Data written to the file successfully")
f.close()
```

abcd.txt:

```
------------
Info
Software
Solutions
```

Note: In the above program, data present in the file will be overridden every time if we are run the program. Instead of overriding if we want to append operation then we should open the file as follows.

```
f=open("abcd.txt", "a")
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

Eg:

```
f=open("abcd.txt","w")
list=["sunny\n","bunny\n","chinny\n","pinny"]
f.writelines(list)
print("List of lines written to the file successfully")
f.close()
```

abcd.txt:

```
sunny
bunny
chinny
pinny
```

Note: While writing data by using write() methods, compulsory we have to provide line separator(\n), otherwise total data should be written in a single line.

D Y PATIL UNIVERSITY
NAVI MUMBAI

Reading character data from the text files:

We can read character data from the text file by using following methods.

- read() ==>To read all data from the file.
- read(n) ==>To read 'n' characters from the file.
- readline() ==>To read only one line.
- readlines() ==>To read all lines into a list.

Ex-1: To read total data from the file

```
f=open("abcd.txt","r")
data=f.read()
print(data)
f.close()
```

o/p:

```
sunny
bunny
chinny
pinny
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

Ex-2:To read only first 10 characters

```
1) f=open("abcd.txt","r")
2) data=f.read(10)
3) print(data)
4) f.close()
```

o/p:
sunny
bunn

Ex-3:To read data line by line

```
1) f=open("abcd.txt",'r')
2) line1=f.readline()
3) print(line1,end='')
4) line2=f.readline()
5) print(line2,end='')
6) line3=f.readline()
7) print(line3,end='')
8) f.close()
```

o/p:
sunny
bunny
chinny

D Y PATIL UNIVERSITY
NAVI MUMBAI

Ex-4:To read all lines into list:

```
1) f=open("abcd.txt","r")
2) lines=f.readlines()
3) print(type(lines))
4) for line in lines:
5)   print(line,end='')
6) f.close()
```

```
o/p:D:\pythonclasses>py test.py
<class 'list'>
sunny
bunny
chinny
pinny
```

Ex-5:

```
1) f=open("abcd.txt","r")
2) print(f.read(3))
3) print(f.readline())
4) print(f.read(4))
5) print("Remaining Data")
6) print(f.read())
7) f.close()
```

```
o/p:D:\pythonclasses>py test.py
sun
ny

bunn
Remaining Data
y
chinny
pinny
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

The with statement:

-->The with statement can be used while opening a file. we can use this to group file operation statements **with** a block.

-->The advantage of **with statement** is it will take care closing of file, after completing all operations automatically even in case of exceptions also, and we are not required to close explicitly.

Eg:

```
with open("abcd.txt","w") as f:
    f.write("Info\n")
    f.write("Software\n")
    f.write("Solutions\n")
print("Is file is closed:",f.closed)
print("Is file is closed:",f.closed)
```

D:\pythonclasses>py test.py

Is file is closed: False

Is file is closed: True

D Y PATIL UNIVERSITY
NAVI MUMBAI

The tell() and seek() methods:

tell():

We can use tell() method to return current position of the cursor(file pointer) from beginning of the file.**[tell current position]**

The position(index) of first character in file is zero just like string index.

Eg:

```
f=open("abc.txt","r")
print(f.tell())
print(f.read(3))
print(f.tell())
print(f.read(5))
print(f.tell())
```

Output

```
0
Inf
3
o
Sof
9
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

seek():

We can use seek() method to move cursor(file pointer)to specified location.

f.seek(offset,fromwhere)

offset represents the number of positions.

->The allowed values for second attribute(from where) are:

0-->From beginning of file(default value)

1-->From current position

2-->From end of the file

Note: Python-2 supports all 3-values but python-3 supports only zero

D Y PATIL UNIVERSITY
NAVI MUMBAI

Example:

```
data="All Students Are GOOD"
f=open("abc.txt",'w')
f.write(data)
with open("abc.txt","r+") as f:
    text=f.read()
    print(text)
    print("The current position:",f.tell())
    f.seek(17)
    print("The current position:",f.tell())
    f.write("GEMS!!!!!")
    f.seek(0)
    text=f.read()
    print("Data After Modification:")
    print(text)
```

Output:

```
All Students Are GOOD
The current position: 21
The current position: 17
Data After Modification:
All Students Are GEMS!!!!!
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

How to check a particular file exists or not?

-->We can use os library to get information about files in our computer.

-->os module has path module, which contains isfile() function to check whether a particular file exists or not?

Syntax: os.path.isfile(filename)

Q:w.a.p to check whether the given file exist or not. If it is available then print its content.

```
import os,sys
fname=input("Enter File Name:")
if os.path.isfile(fname):
   print("File Exists:",fname)
   f=open(fname,'r')
else:
   print("File does not exist",fname)
   sys.exit(0)
print("The content of file is:")
data=f.read()
print(data)
```

```
Enter File Name:a.txt
File does not exist a.txt

============== RESTART: C:/S
Enter File Name:abc.txt
File Exists: abc.txt
The content of file is:
All Students Are GEMS!!!!!
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

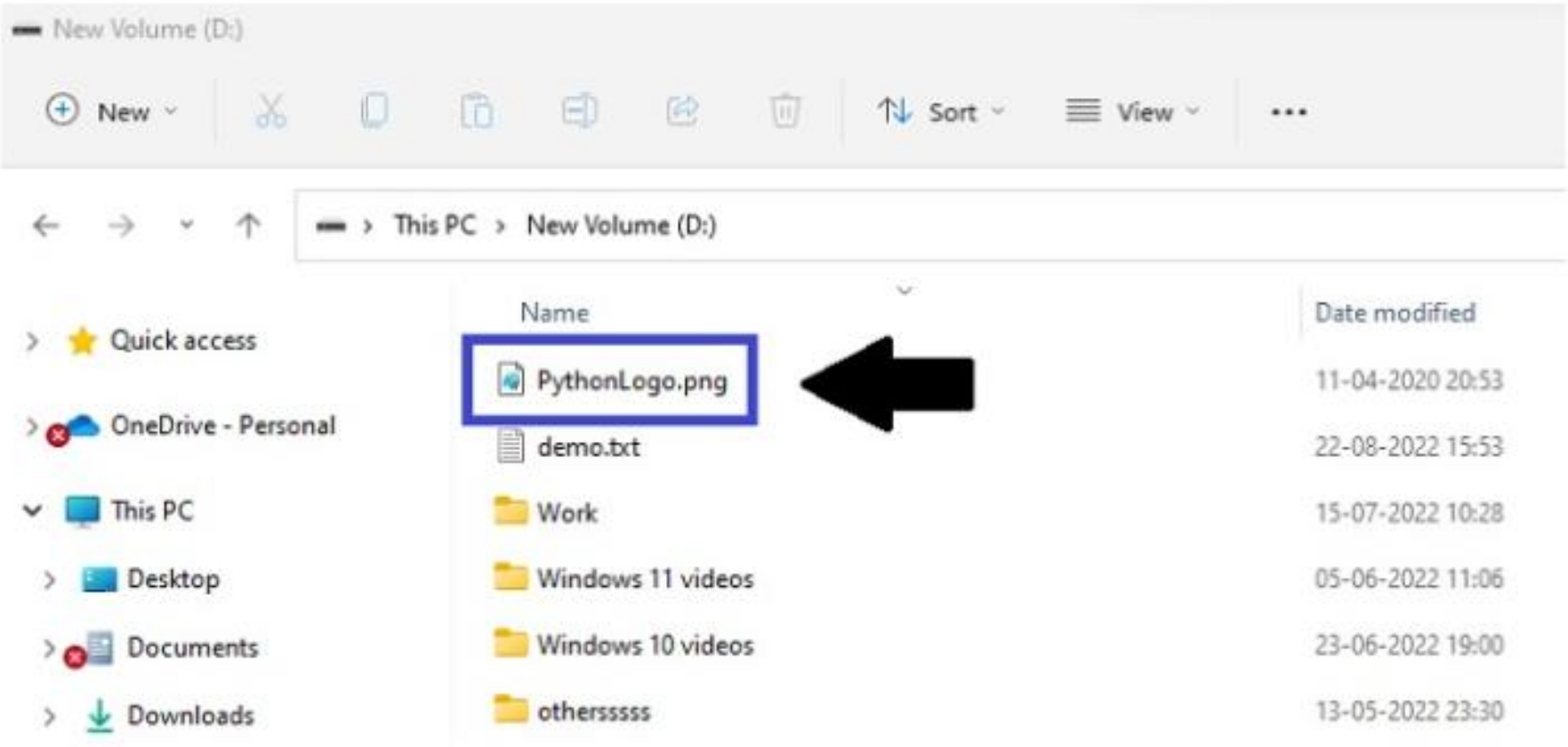
Reading and Writing to Binary Files

To read or write a binary file, at first you need to understand the different file modes for Binary Files in Python ?

Mode	Description
rb	Opens a file for reading only in binary format. The file pointer is placed at the beginning of the file. This is the default mode.
rb+	Opens a file for both reading and writing in binary format. The file pointer placed at the beginning of the file.
wb	Opens a file for writing only in binary format. Overwrites the file if the file exists. If the file does not exist, creates a new file for writing.
wb+	Opens a file for both writing and reading in binary format. Overwrites the existing file if the file exists. If the file does not exist, creates a new file for reading and writing.
ab	Opens a file for appending in binary format. The file pointer is at the end of the file if the file exists. That is, the file is in the append mode. If the file does not exist, it creates a new file for writing.
ab+	Opens a file for both appending and reading in binary format. The file pointer is at the end of the file if the file exists. The file opens in the append mode. If the file does not exist, it creates a new file for reading and writing.

Read a Binary File

Let's say we have a binary file. We can read it using the read() method. First, we will open the Binary File using the rb mode of the open() method. We have the following source file in D: drive ?



D Y PATIL UNIVERSITY NAVI MUMBAI

Example

Let's see the complete code

```
# Open a binary file
f = open('D:\PythonLogo.png', 'rb')

# Read lines
data = f.read()

# Display the data
print(data)
```

Output

The Binary Files are not huma-readable and the content is unrecognizable. The actual output was bigger. We have shown only some part below.

```
b'\x89PNG\r\n\x1a\n\x00\x00\x00\rIHDR\x00\x00\x04\xb0\x00\x00\x04\xb0\x08\x06\x
\xeb!\xb3\xcf\x00\x00\x00\x04gAMA\x00\x00\xb1\x8f\x0b\xfca\x05\x00\x00\x00
cHRM\x00\x00z&\x00\x00\x80\x84\x00\x00\xfa\x00\x00\x00\x80\xe8\x00\x00u0\x00\x
```


TIL
ITY
NAVI MUMBAI

Write to a Binary File

The **wb** mode of the open() method is used to open a file in format form writing.

Note ? The Binary Files are not huma-readable and the content is unrecognizable

Let's see the complete example. Here, the file will get stored in **E drive** with the name

Example

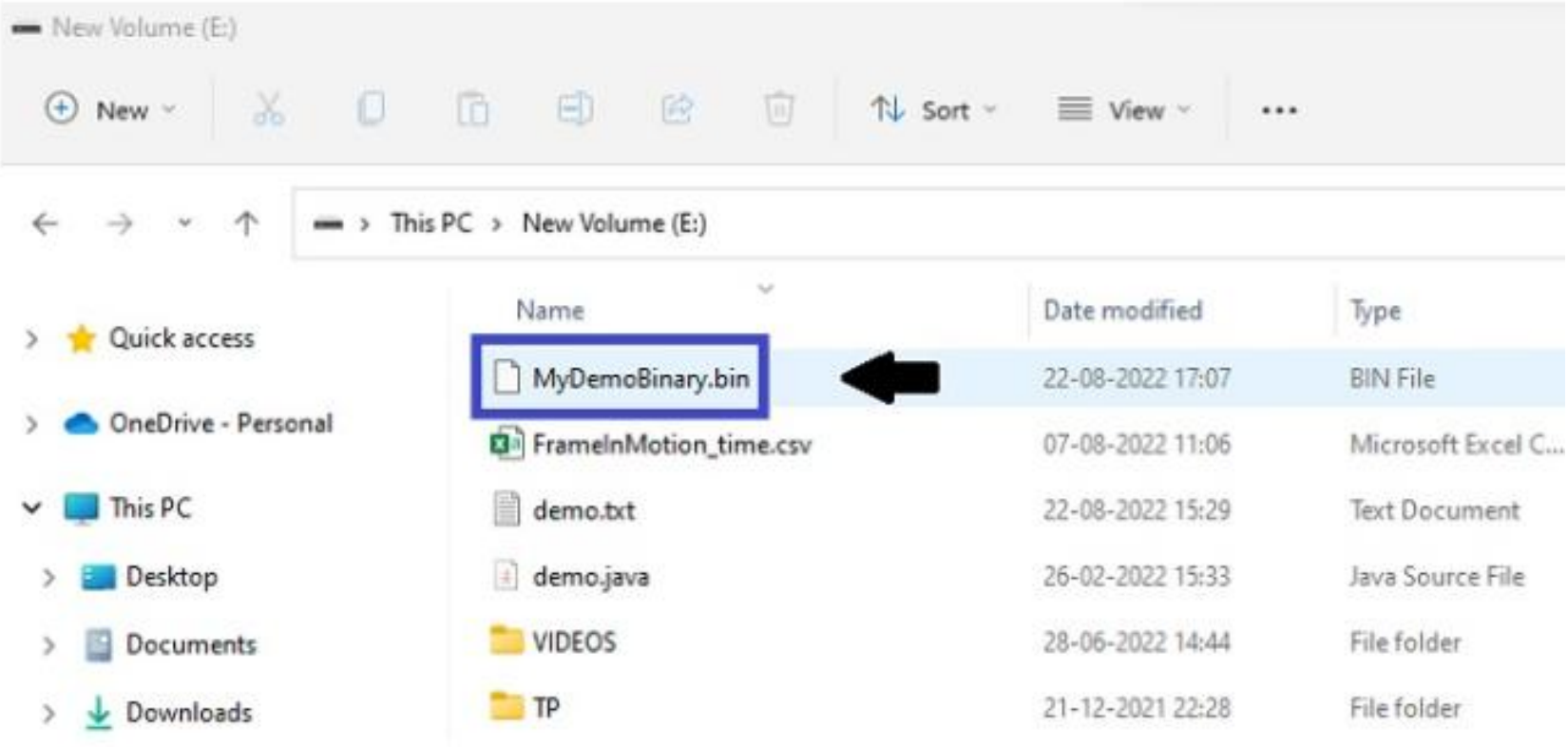
MyDemoBinary.bin ?

```
# Open a file in binary format for writing
f = open("E:\MyDemoBinary.bin","wb")

# Elements to be added to the binary file
a = [100, 200, 300]

# Convert the integer elements to a bytearray
myArr = bytearray(a)

# The byte representation ius now written to the file
f.write(myArr)
f.close()
```


Y PATIL
NIVERSITY
NAVI MUMBAI

Output

After running the file, the file is visible in the set path i.e. E drive ?



D Y PATIL UNIVERSITY
NAVI MUMBAI

Handling Binary Data with read and write:

->It is very common requirement to read or write binary data like images, videos files , audio files etc....

Q:w.a.p to read image file and write to a new image file.

```
f1=open("sunny.jpg",'rb')
f2=open("newpic.jpg",'wb')
bytes=f1.read()
f2.write(bytes)
print("New image is available with the name:newpic.jpg")
```

Output:

```
============ RESTART: C:/Swati/pythonCode/binaryDemo.
New image is available with the name:newpic.jpg
```

Note: Save Sunny.jpg file in same folder where file save

D Y PATIL UNIVERSITY
NAVI MUMBAI

Reading and Writing CSV Files in Python

- **CSV (Comma Separated Values)** format is one of the most widely used formats for storing and exchanging structured data between different applications, including databases and spreadsheets.
- CSV files store tabular data, where each data field is separated by a delimiter, typically a comma.
- Python provides built-in support for handling CSV files through the csv module, making it easy to read, write and manipulate CSV data efficiently.

D Y PATIL UNIVERSITY
NAVI MUMBAI

Writing to a csv file in python

Python provides the **csv.writer class** to write data to a CSV file. It converts user data into delimited strings before writing them to a file. While opening the file, using newline='' prevents unnecessary newlines when writing.

Syntax:

```
csv.writer(csvfile, dialect='excel', **fmtparams)
```

Parameters:

- **csvfile:** The file object where CSV data will be written.
- **dialect:** Defines a set of parameters to control CSV writing (default is 'excel').
- ****fmtparams:** Additional formatting options such as delimiter, quotechar,etc.

Writing a single row:

```
writerow(fields)
```

Writing multiple rows:

```
writerows(rows)
```

Writing to a csv file in python

Eg:

```
import csv
with open("emp.csv","w",newline='') as f:
    w=csv.writer(f)#Returns csv writer object pointing to f
    w.writerow(["ENO",'ENAME',"ESAL","EADDR"])
    n=int(input("Enter Number of Empolyees:"))
    for i in range(n):
        eno=input("Enter Employee No:")
        ename=input("Enter Employee Name:")
        esal=input("Enter Employee Sal:")
        eaddr=input("Enter Employee Address:")
        w.writerow([eno,ename,esal,eaddr])
        print("Total employees data written to csv file successfully")
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

Output:

```
Enter Number of Empolyees:2
Enter Employee No:1
Enter Employee Name:swati
Enter Employee Sal:7000000
Enter Employee Address:cbd
Total employees data written to csv file successfully
Enter Employee No:2
Enter Employee Name:prachi
Enter Employee Sal:8709000
Enter Employee Address:Vashi
Total employees data written to csv file successfully
```

Note: Observe the difference with newline attribute and without.

```
with open("emp.csv","w",newline='') as f:
with open("emp.csv","w") as f:
```

Note:

- If we are not using newline attribute then in the csv file blank lines will be included between data.
- To prevent these blank lines, newline attribute is required in python-3, but in python-2 just we can specify mode as 'wb' and we are not required to use newline attribute.

D Y PATIL UNIVERSITY
NAVI MUMBAI

Reading data from csv file:

- To read a CSV file, Python provides the csv.reader class, which reads data in a structured format.
- The first step involves opening the CSV file using the open() function in read mode ('r').
- The **csv.reader() function** then reads the file, returning an iterable reader object.
- **Syntax:**

```
csv.reader(csvfile, dialect='excel', **fmtparams)
```

- **Parameters:**

csvfile : The file object containing CSV data.

dialect : Defines a set of parameters to control CSV reading (default is 'excel').

****fmtparams:** Additional formatting parameters like delimiter, quotechar, etc.

D Y PATIL UNIVERSITY
NAVI MUMBAI

Reading data from csv file:

```
import csv
f=open("emp.csv","r")
r=csv.reader(f)#Returns csv reader object
print(type(r))
data=list(r)
#print(data)
for line in data:
    for word in line:
        print(word,"\t",end='')
    print()
```

Output:

```
<class '_csv.reader'>
ENO     ENAME   ESAL    EADDR
1       swati   7000000         cbd
2       prachi  8709000         Vashi
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

Example:

```python
import csv

# Opening the CSV file
with open('Giants.csv', mode='r') as file:
    # Reading the CSV file
    csvFile = csv.reader(file)

    # Skipping the header (uncomment if needed)
    # next(csvFile)

    # Displaying the contents of the CSV file
    for line in csvFile:
        print(line)
```

Example CSV File: Giants.csv

	A	B	C
1	name	age	grade
2	Steve	13	A
3	John	14	F
4	Nancy	14	C
5	Ravi	13	B
6			

.CSV file

Output

```
['Steve', 13, 'A']
['John', 14, 'F']
['Nancy', 14, 'C']
['Ravi', 13, 'B']
```

Explanation: The **csv module** reads **Giants.csv** using **csv.reader(file),** iterating row by row. The **with statement** ensures proper file closure. Each row is returned as a list of column values. Use **next(csvFile)** to skip the header.

D Y PATIL UNIVERSITY
NAVI MUMBAI

Thank You

D Y PATIL
UNIVERSITY
NAVI MUMBAI